

noip2020LH模拟赛题解

by chenguowen01 from YYHS

2020 年 9 月 20 日

T1:

算法 1:

直接暴搜。

可以过掉 subtask1, 15 分。

算法 2:

每次暴力 dp, 复杂度 $\Theta(qnm)$ 。

可以过掉 subtask1 和 subtask2, 共 30 分。

算法 3:

考虑到合并两个背包复杂度为 $\Theta(m^2)$ 。

所以用线段树之类的数据结构维护可以做到 $\Theta(nm^2 + qm^2 \lg n)$ 。

可以过 subtask1 和 subtask4, subtask2 可能能过, 35~50 分。

算法 4:

用分块或者莫队可以做到每次只往背包里增加一个元素, 这样复杂度变为 $\Theta((n+q)m\sqrt{n})$ 。

可以过 subtask1, 2, 3, 4, 可能能过 subtask5, 60~75 分。

算法 5:

我们有两种背包的操作可以做到复杂度 $O(m)$: 一种是往背包里加入一个元素, 一种是询问两个背包合并后价格总和不超过某个定值的答案。

我们可以考虑分治, 对于一段背包的区间 $[l, r]$, 如果足够小, 我们可以暴力所有的询问。

否则可以取中点 $mid = \lfloor \frac{l+r}{2} \rfloor$, 从 mid 往两边分别做前缀/后缀背包, 那么此时每个过中点的询问可以视为两个已处理出来的背包合并后询问价格总和不超过 m 的答案, 可以直接 $O(m)$ 解决。

那么此时没有过中点的询问，所以可以分治解决 $[l, m]$ 和 $[m + 1, r]$ 。

复杂度 $\Theta(nm \lg n + qm)$ ，100 分。

T2:

算法 1:

根据题意写代码, $\Theta(n^4)$ 或 $\Theta(n^3)$ 。

可以过 subtask1, 8 分。

算法 2:

由于最近公共祖先具有结合律 (即 $\text{lca}(\text{lca}(x, y), z) = \text{lca}(x, \text{lca}(y, z))$), 所以我们可以扫描的时候顺便把前面若干项的最近公共祖先记录下来, 再和新加入的做 lca , 然后我们再随便写个求 lca 的算法。

复杂度 $\Theta(n^2 \lg n)$ 或者 $\Theta(n^2)$ 。

可以过 subtask1,2, 20 分。

算法 3:

考虑每个点作为 j 产生的贡献, 那么如果有 i 要在 j 节点产生贡献, 那么 $[j, i]$ 中的所有元素都要在 j 的子树内。所以能从 j 产生贡献的 i 是一段区间。我们可以用平衡树启发式合并或者线段树合并 + 二分来求解, 然后进行区间修改。

复杂度 $\Theta(n \lg^2 n)$

可以过 subtask1~4, 可能能过 subtask5,6,7, 52~84 分。

算法 4:

算法 3 用线段树合并再用线段树上二分之类的可以做到 $\Theta(n \lg n)$

可以过 subtask1~7, 可能能过 subtask8, 84~100 分。

算法 5:

整理一下发现区间修改可以用差分, 求解区间相当于求最小的编号比它大的不在它子树内的成员, 可以记下 dfs 序然后整体开线段树二分, 不需要线段树合并。

可以 AC。

算法 6:

可以发现算法 5 的做法可以用单调栈维护。复杂度 $\Theta(n)$ ，可以 AC。

T3:

算法 1:

暴搜每一棵树。

能过 subtask1, 或许能过 subtask2, 12~24 分。

算法 2:

考虑 subtask4, 相当于新的方案所有边权全是 1, 然后随便连。直接组合数计算即可。

复杂度 $\Theta(n^2)$, 能过 subtask4, 8 分。

算法 3:

对于 subtask3。就相当于求合法的生成树方案数

考虑这个生成树的结构, 发现就相当于确定 kruskal 重构树求原树的方案。

我们先求出 kruskal 重构树。然后每一个节点就相当于将其几个儿子对应的树按某种方式用边权等于这个节点权值的边接起来。

如果一个节点的儿子的树中分别有 $a_1, a_2, \dots, a_m$ 个节点, 那么这些树接起来的方案为:

$$\prod_{1 \leq i \leq m} a_i \left(\sum_{1 \leq i \leq m} a_i \right)^{m-2}$$

然后直接暴力算即可。

复杂度 $\Theta(n^2)$, 能过 subtask3, 16 分。

算法 4:

根据算法 3, 我们可以进行树形 dp。对于每个节点:

1. 如果该节点子树内 u, v 都在, 只记录对于 u, v 的每种距离的方案数。
2. 有 u, v 其中一个, 设其为 w , 那么只记录对于该子树中 w 到其他任意一个节点每种距离的方案数。
3. u, v 中的点一个都没有, 记录该子树中任意点到任意点的每种距离的方案数。

那么转移可以暴搜每棵树在不在关键链上，复杂度 $\Theta(2^n \times n^3)$ 。

可以过 subtask1~2, 24 分。

算法 5:

继续考虑算法 4 的 dp，我们可以在转移的时候再写一个 dp，依次考虑每棵树是否加入关键链。那么我们根据生成树计数公式，我们只需要记录关键链里接入的树的个数以及这些树节点个数的总和即可。复杂度 $O(n^5)$

能过 subtask1,2,5, 可能能过 subtask6, 40~48 分。

综合之前的算法 2 和算法 3 可以拿到 64~72 分。

算法 6:

对之前的 $O(n^5)$ 的 dp 进行一些优化，可以优化到 $O(n^3 \lg^2 n)$ 或者一些其他的玄学复杂度。

能过 subtask1,2,5,6, 可能能过 subtask7, 48~60 分。

算法 7:

考虑去掉一维 dp 状态，显然树的个数不能省，那么可以考虑省掉树的 size 和。我们可以拆开生成树计数公式，设关键链包含的树大小分别为 $c_1, c_2, \dots, c_k$ ，那么正常情况下我们会产生的贡献为：

$$\frac{\sum_{i=1}^k c_i}{\prod_{i=1}^k c_i} = \sum_{i=1}^k \frac{c_i}{\prod_{j=1}^k c_j}$$

相当于这些树里面取任意一个产生贡献，然后加起来。那么我们就可以去掉 size 和的这一维，改成一维 0/1 表示是否有树贡献过。这样复杂度就变为了 $\Theta(n^3)$

可以 AC, 100 分。

算法 8:

可以用分治 FFT 优化算法 7, $\Theta(n^2 \lg^2 n)$ 。出题人没写过，您可以试试。

T4:**算法 1:**

根据题意暴力。

可以通过 subtask1, 16 分。

算法 2:

考虑 subtask2, $k = 1$ 时根据二项式定理答案为 $(1 + ab)^n$

直接快速幂即可。

可以通过 subtask2, 8 分。

算法 3:

利用单位根和二项式定理求解, 最优可以做到 $O(k^2 \lg k (\lg n + \lg k))$ 。

可以通过 subtask2 5, 56 分。

算法 4:

令 $z^k = a$ 考虑 $(1 + zb)^n$, 我们可以在运算中将 $z^k = a$ 直接带入, 其他 $z^i (0 \leq i < k)$ 的项保留。那么答案就是 $(1 + zb)^n$ 最后不带 z 的项。显然这个符合结合律, 所以直接快速幂 + FFT 即可。

可以 AC。